

XSS 漏洞报告 (Lab2: 将 XSS 存储为锚属性, 并用 HTML 编码双引号 href)

1. 漏洞名称

跨站脚本攻击 (Cross-Site Scripting, XSS) — 存储型 XSS (javascript: 伪协议注入)

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站的评论功能存在存储型 XSS 漏洞。在提交评论时, 前端页面没有对“网站”输入框的内容进行严格的过滤和转义, 导致攻击者可以在评论中注入 javascript:alert(1) 这种伪协议链接。当其他用户点击该评论者的名字 (对应的 标签) 时, 恶意 JS 代码便会执行。

4. 漏洞位置

请求方式: POST /post/comment

漏洞参数: website (评论表单中的“网站”输入框)

5. 漏洞复现过程

第一步: 正常提交与发现反射点

在博客文章下方, 填写评论、姓名、邮箱, 并在“网站”输入框填写普通测试字符串 (如 test)。提交后, 页面显示“谢谢你的评论!”, 返回博客详情页, 发现刚才填写的“网站”字符串变成了一个可点击的链接。

第二步: 构造特殊 Payload 并提交

“网站”输入框在底层会被渲染为 HTML 的 标签。为了触发恶意代码, 构造 javascript: 伪协议 链接。

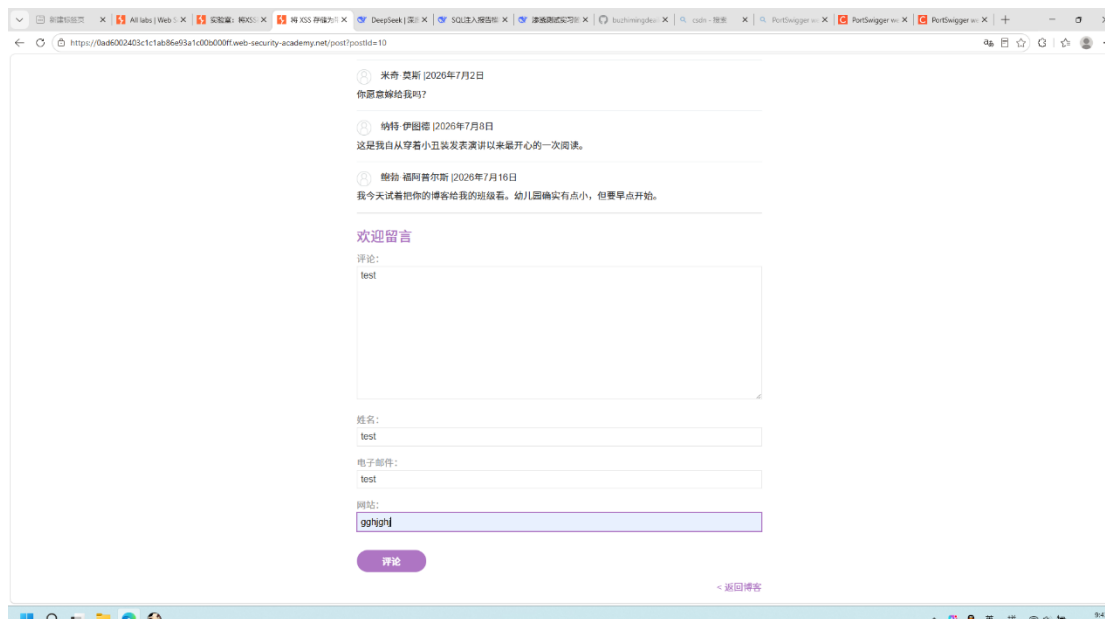
在“网站”输入框中输入 Payload:

text

javascript:alert(1)

填写其他必填项, 点击“评论”提交。

图 1: 在评论区提交包含恶意 Payload 的表单

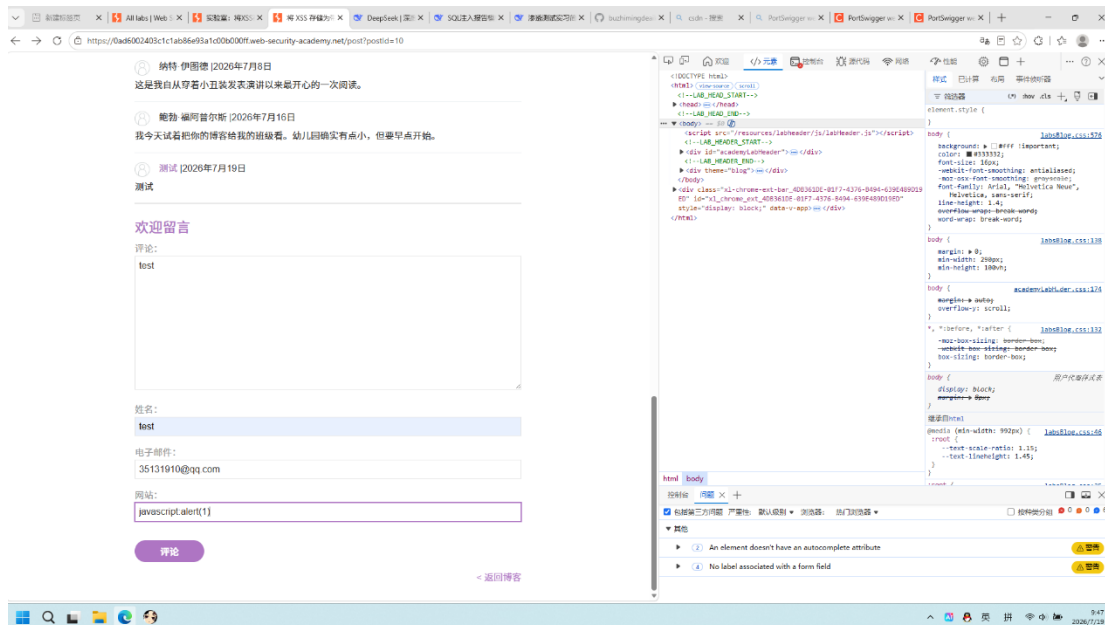


说明：利用 javascript: 伪协议，将恶意执行代码伪装成链接地址。

第三步：发现 Payload 被存储且未被完全过滤

提交成功后，再次刷新或回到博客页面。使用浏览器开发者工具（F12）查看“元素”面板，找到评论者名字对应的 HTML 代码。

图 2：审查源代码发现注入成功

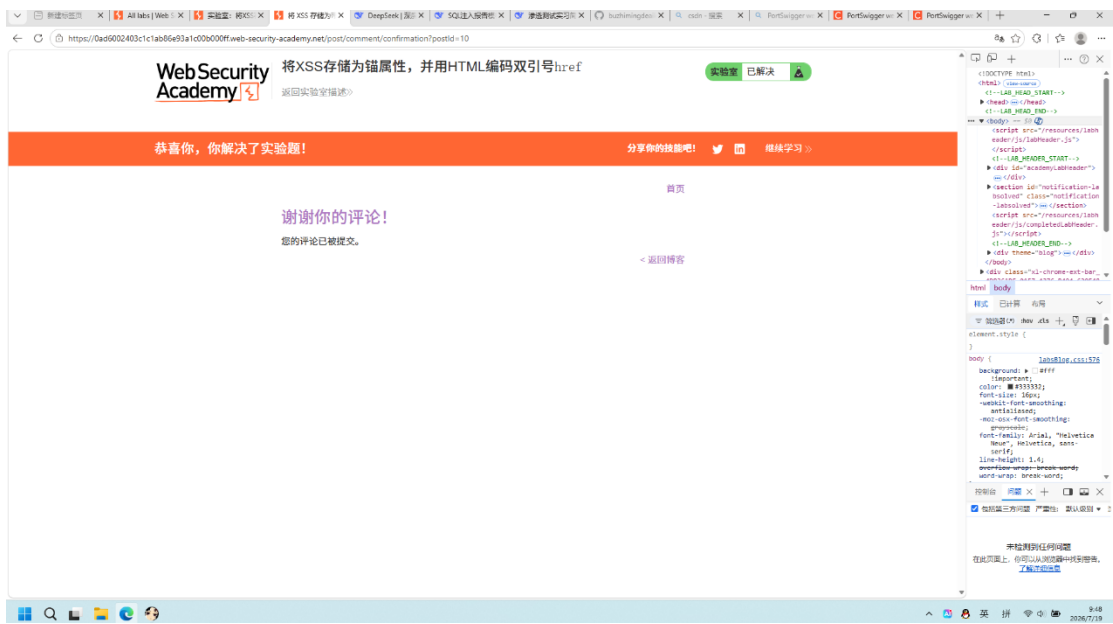


说明：后端在存储该链接时，没有对 javascript: 协议进行黑名单过滤。恶意代码已经被成功保存到了服务器数据库中。

第三步：点击触发 Payload

在博客评论列表中，用鼠标点击刚才发表评论的“姓名”（这里显示为 test）。

图 3：成功触发 XSS 并完成实验



说明：点击评论者的名字后，浏览器成功执行了存储在数据库中的 javascript:alert(1)，漏洞利用链完全打通。

6. 漏洞原理

该漏洞属于存储型 XSS 结合 URL 伪协议利用：

- 1. 攻击者提交：在评论的“网站”栏中输入 javascript:alert(1)。
- 2. 服务器存储：后端服务器未对 javascript: 关键字进行过滤，将其直接存入数据库。
- 3. 受害者渲染：当浏览器加载该页面时，评论姓名被渲染为 姓名。
- 4. 交互触发：由于 <a> 标签的 href 属性天然支持 javascript: 伪协议，当用户点击该超链接时，浏览器会直接执行 javascript: 后面的恶意 JS 代码。

7. 漏洞影响

- 持久性危害：这是一种典型的存储型 XSS，恶意代码被永久留在数据库里。任何访客（包括管理员）只要浏览这篇博客，都有被攻击的风险。
- 权限升级：如果管理员点击了该链接，攻击者就能窃取管理员的高权限 Cookie 甚至直接操控后台。

8. 修复建议

① 对 href 属性值进行安全过滤：后端在处理“网站”等链接参数时，不应仅仅做 HTML 编码（转义双引号），还必须校验 URL 协议白名单。只允许 http://、https://，严格禁

止 javascript:、data:、vbscript: 等危险协议。

② **使用** rel="noreferrer" 和 rel="noopener": 对于外部链接, 增加安全属性以降低风险。

③ **内容安全策略 (CSP)**: 配置 script-src 'self', 防止页面执行内联的 JavaScript 代码。

9. 工具

- **Google Chrome / Firefox** (浏览器开发者工具、元素审查)
- **PortSwigger Web Security Academy** (靶场平台)